Algorithms 2

Reductions

Daniel Rosenberg & Michael Trushkin

Ariel University

1 NP-completeness

1.1 NP-completeness

- As we saw, most problems are *hard*!

1.1 NP-completeness

- As we saw, most problems are *hard*!

Definition 1.1.1 (NP class) A language L is said to be in $\mathbf{NP}$ if we have a polynomial-time algorithm M such that

$$x \in L \Leftrightarrow \exists y \ s.t \ |y| < p(|x|) \text{ and } M(x, y) = 1$$

where p is some polynomial

- In most literture y is called a *witness* and M is called *veryfing algorithm*, where y plays the role of the answer, and M should just verify if the answer is correct.

1.1 NP-completeness

- As we saw, most problems are *hard*!

Definition 1.1.1 (NP class) A language L is said to be in $\mathbf{NP}$ if we have a polynomial-time algorithm M such that

$$x \in L \Leftrightarrow \exists y \ s.t \ |y| < p(|x|) \text{ and } M(x, y) = 1$$

where p is some polynomial

- In most literture y is called a *witness* and M is called *veryfing algorithm*, where y plays the role of the answer, and M should just verify if the answer is correct.
- We talked about a few:
  - k-clique
  - CNF-SAT
  - k-CNF-SAT

2 Reductions

2.1 Reductions

- We saw we can “translate” one languge into another with reductions

2.1 Reductions

- We saw we can “translate” one languge into another with reductions

Definition 2.1.1 (polynomial time reduction) Given two languages $L_1, L_2 \in \mathbf{NP}$, we write $L_1 \leq_p L_2$ if there exists a function $f : \{0,1\}^* \to \{0,1\}^*$ and a polynomial $p : \mathbb{N} \to \mathbb{N}$, such that:

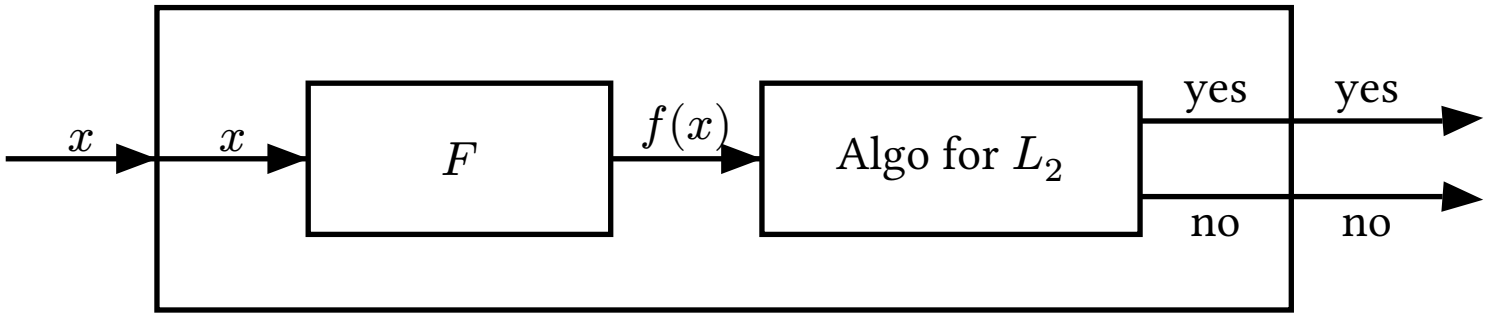
- $x \in L_1 \Leftrightarrow f(x) \in L_2$
- for every $x \in \{0,1\}^*$, f runs in $p(|x|)$ time.

2.1 Reductions

- We saw we can “translate” one languge into another with reductions

Definition 2.1.1 (polynomial time reduction) Given two languages $L_1, L_2 \in \mathbf{NP}$, we write $L_1 \leq_p L_2$ if there exists a function $f : \{0,1\}^* \to \{0,1\}^*$ and a polynomial $p : \mathbb{N} \to \mathbb{N}$, such that:

- $x \in L_1 \Leftrightarrow f(x) \in L_2$
- for every $x \in \{0,1\}^*$, f runs in $p(|x|)$ time.



2.2 Hard Langauges

- For any complexity class, there are hard languages

Definition 2.2.1 (NP-hard) A language $L \subseteq \{0,1\}^*$ is said to be NP-hard if $L' \leq_p L$ for every $L' \in \mathbf{NP}$

Definition 2.2.2 (NP-complete) A language $L \subseteq \{0,1\}^*$ is said to be NP-complete if $L \in \mathbf{NP}$ and L is NP-hard

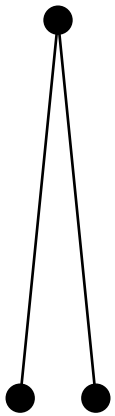
- We saw that CNF-SAT and k-CNF-SAT are NP Complete

3 Independent set

3.1 Independent set

For a graph G, verteces $v_1, v_2, v_3 \ldots \subseteq V(G)$ are called *indepedent* if there is no edges between them.

indepedent



not indepedent

3.1 Independent set

- For a graph G, let $\alpha(G)$ denote the size of the maximum independent set in G.

Definition 3.1.1 IS $:= \{< G, k >: \alpha(G) \geq k\}$.

Theorem 3.1.2 IS is in $\mathbf{NPC}$.

3.2 Independent set

- The proof that IS is in **NP** is left as homework.
- In order to show that IS is hard it is enough to show that

$$L \leq_p \text{IS}$$

for some lanaguege L where $L \in \mathbf{NPH}$.

- Here, we show that

$$\text{3-CNF-SAT} \leq_p \text{IS}$$

Claim 3.2.1 Let $L \in \mathbf{NPH}$, and let L' be a language. If $L \leq_p L'$, then L' is also **NPH**.

3.3 3-CNF-SAT $\leq_p$ IS

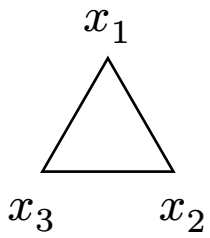
- Given a 3-CNF formula φ, construct a graph G_φ as follows:
  1. **Triangles**: For each clause l_1, l_2, l_3 we create a triangle with 3 vertices named $v_{l_1}, v_{l_2}, v_{l_3}$.
  2. **Consistency Edges**: For any pair of complementary literals $x_j, \overline{x_j}$ that are in different clauses, put an edge between the vertices that correspond to the literals.

$$\varphi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3)$$

3.3 3-CNF-SAT $\leq_p$ IS

- Given a 3-CNF formula φ, construct a graph G_φ as follows:
  1. **Triangles**: For each clause l_1, l_2, l_3 we create a triangle with 3 vertices named $v_{l_1}, v_{l_2}, v_{l_3}$.
  2. **Consistency Edges**: For any pair of complementary literals $x_j, \overline{x_j}$ that are in different clauses, put an edge between the vertices that correspond to the literals.

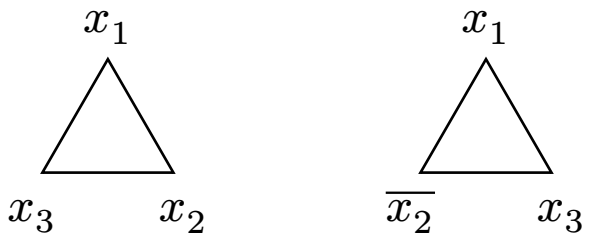
$$\varphi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3)$$



3.3 3-CNF-SAT $\leq_p$ IS

- Given a 3-CNF formula φ, construct a graph G_φ as follows:
  1. **Triangles**: For each clause l_1, l_2, l_3 we create a triangle with 3 vertices named $v_{l_1}, v_{l_2}, v_{l_3}$.
  2. **Consistency Edges**: For any pair of complementary literals $x_j, \overline{x_j}$ that are in different clauses, put an edge between the vertices that correspond to the literals.

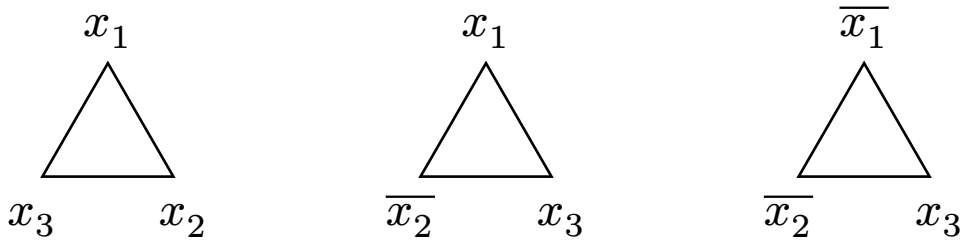
$$\varphi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3)$$



3.3 3-CNF-SAT $\leq_p$ IS

- Given a 3-CNF formula φ, construct a graph G_φ as follows:
  1. **Triangles**: For each clause l_1, l_2, l_3 we create a triangle with 3 vertices named $v_{l_1}, v_{l_2}, v_{l_3}$.
  2. **Consistency Edges**: For any pair of complementary literals $x_j, \overline{x_j}$ that are in different clauses, put an edge between the vertices that correspond to the literals.

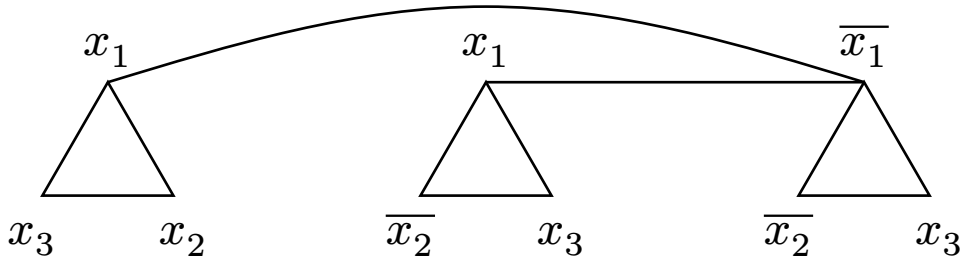
$$\varphi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3)$$



3.3 3-CNF-SAT $\leq_p$ IS

- Given a 3-CNF formula φ, construct a graph G_φ as follows:
  1. **Triangles**: For each clause l_1, l_2, l_3 we create a triangle with 3 vertices named $v_{l_1}, v_{l_2}, v_{l_3}$.
  2. **Consistency Edges**: For any pair of complementary literals $x_j, \overline{x_j}$ that are in different clauses, put an edge between the vertices that correspond to the literals.

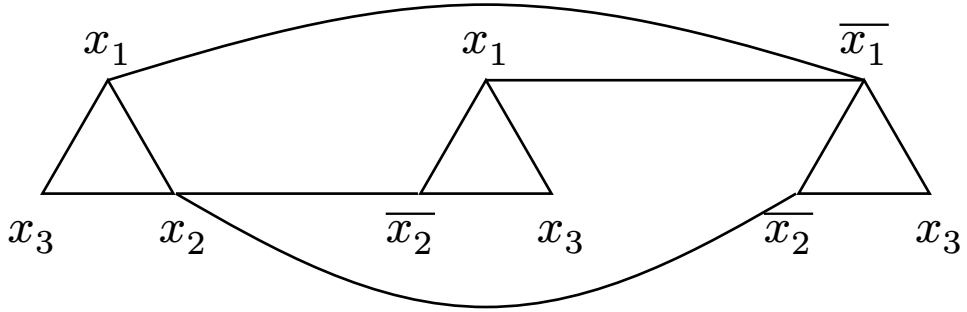
$$\varphi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3)$$



3.3 3-CNF-SAT $\leq_p$ IS

- Given a 3-CNF formula φ, construct a graph G_φ as follows:
  1. **Triangles**: For each clause l_1, l_2, l_3 we create a triangle with 3 vertices named $v_{l_1}, v_{l_2}, v_{l_3}$.
  2. **Consistency Edges**: For any pair of complementary literals $x_j, \overline{x_j}$ that are in different clauses, put an edge between the vertices that correspond to the literals.

$$\varphi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3)$$



- return the pair $< G_\varphi, m >$ where m is the number of clauses.

3.4 Independent set

- The algorithm runs in poly time(why?)
- Now we need to prove that

$$\varphi \in \text{3-CNF-SAT} \iff < G_\varphi, m > \in \text{IS}$$

$\Longrightarrow$:

- Let φ be satisfiable and let α_φ be a satisfying assignment for φ.
- As α_φ satisfies φ, at least one literal of each clause is satisfied, pick any one such literal from each clause.
- The set of vertices corresponding to the set of literals chosen is independent in G_φ and has size of m.

3.5 Independent set

- The algorithm runs in poly time(why?)
- Now we need to prove that

$$\varphi \in \text{3-CNF-SAT} \iff < G_\varphi, m > \in \text{IS}$$

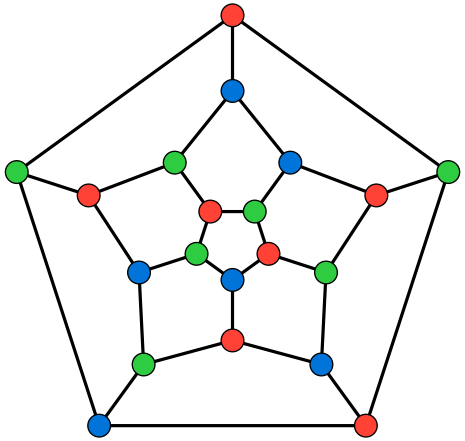
$\Longleftarrow$:

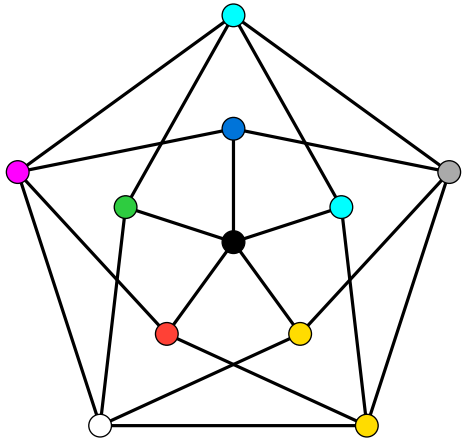
- Suppose G_φ has an independent set of size m, and let S be such an independent set.
- Let $l_1, ..., l_m$ denote the vertices of S,
- for each $i \in m$ let $v(l_i) \in \{x_1, \overline{x_1}, ..., x_n, \overline{x_n}\}$ be the variable corresponding to l_i.
- Set $v(l_i) = T$ for all $i \in [m]$, Finally if $x_i \in [n]$ did not recieve an assigment set $x_i = T$.
- The asssignment is satisfying:
  - every triangle has one vertex in $S \Rightarrow$ every clause has one positive literal
- The assignment is *consistent*:
  - if for some variable $x_i = \overline{x_i}$ then

$$\exists j, k \in [m] \text{ s.t. } v(l_j) = x_i \text{ and } v(l_k) = \overline{x_i} \Rightarrow l_j l_k \in E\left(G_\varphi\right) \Rightarrow S \text{ is not independent.}$$

4 Graph coloring

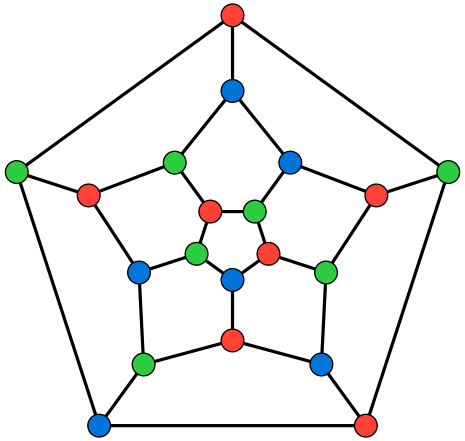
4.1 Graph coloring

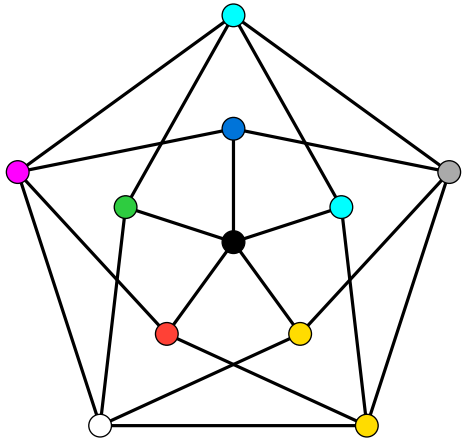




- For a graph G denote by $\chi(G)$ the least $k \in \mathbb{N}$ such that G is k-colorable.

4.1 Graph coloring



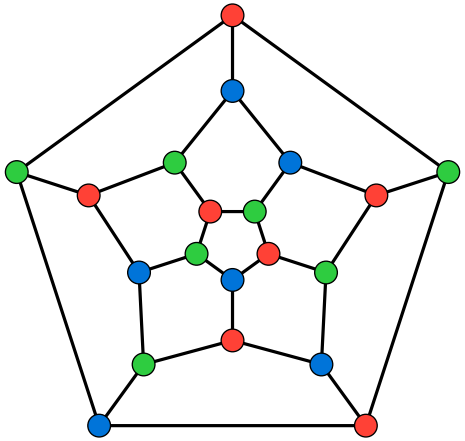


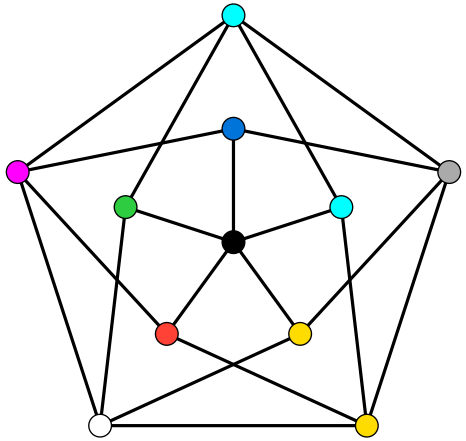
- For a graph G denote by $\chi(G)$ the least $k \in \mathbb{N}$ such that G is k-colorable.

Definition 4.1.1 k-COL $:= \{G : \chi(G) \leq k\}$

It is well known that 2-COL$\in \mathbf{P}$.

4.1 Graph coloring





- For a graph G denote by $\chi(G)$ the least $k \in \mathbb{N}$ such that G is k-colorable.

Definition 4.1.1 k-COL $:= \{G : \chi(G) \leq k\}$

It is well known that 2-COL$\in \mathbf{P}$.

Theorem 4.1.2 3-COL $\in \mathbf{NPC}$.

- The proof that 3-COL is in $\mathbf{NP}$ is left as homework

4.2 NEA-k-CNF-SAT

- Let φ be a formula

4.2 NEA-k-CNF-SAT

- Let φ be a formula
- φ is said to be *not all equal satisfiable* (NAE-SAT) if it has a satisfying assigmnet such that in each caluse it has at least one satisfied literal and at least one that is not satisfied.

4.2 NEA-k-CNF-SAT

- Let φ be a formula
- φ is said to be *not all equal satisfiable* (NAE-SAT) if it has a satisfying assigmnet such that in each caluse it has at least one satisfied literal and at least one that is not satisfied.

Definition 4.2.1 (NAE-k-CNF-SAT)

$$\text{NAE-}k\text{-CNF-SAT} := \{\varphi : \varphi \text{ is NAE-SAT, with exactly k literals in each clause}\}.$$

4.2 NEA-k-CNF-SAT

- Let φ be a formula
- φ is said to be *not all equal satisfiable* (NAE-SAT) if it has a satisfying assigmnet such that in each caluse it has at least one satisfied literal and at least one that is not satisfied.

Definition 4.2.1 (NAE-k-CNF-SAT)

$$\text{NAE-}k\text{-CNF-SAT} := \{\varphi : \varphi \text{ is NAE-SAT, with exactly k literals in each clause}\}.$$

- We are going to show

$$\text{NAE-3-CNF-SAT} \leq_p \text{3-COL}.$$

Remark

In the t.a session you will prove that

$$\text{3-CNF-SAT} \leq_p \text{NAE-3-CNF-SAT}$$

concluding the proof.

4.3 NAE-3-CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula φ, define G_φ as follows:

4.3 NAE-3-CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula φ, define G_φ as follows:

1. Start with a single vertex D. This is our *Don't care vertex*.

4.3 NAE-3-CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula φ, define G_φ as follows:

1. Start with a single vertex D. This is our *Don't care vertex*.
2. For each variable x_i of φ, add two "original" new vertices $x_i, \overline{x_i}$, add an edge between them, and connect both to D. This are our *variable gadgets*.

4.3 NAE-3-CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula φ, define G_φ as follows:

1. Start with a single vertex D. This is our *Don't care vertex*.
2. For each variable x_i of φ, add two "original" new vertices $x_i, \overline{x_i}$, add an edge between them, and connect both to D. This are our *variable gadgets*.
3. For each clause $l_1 \vee l_2 \vee l_3$ we create a triangle with 3 vertices named l_1, l_2, l_3, this is our *clause gadget*.

4.3 NAE-3-CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula φ, define G_φ as follows:

1. Start with a single vertex D. This is our *Don't care vertex*.
2. For each variable x_i of φ, add two "original" new vertices $x_i, \overline{x_i}$, add an edge between them, and connect both to D. This are our *variable gadgets*.
3. For each clause $l_1 \vee l_2 \vee l_3$ we create a triangle with 3 vertices named l_1, l_2, l_3, this is our *clause gadget*.
4. For each literal in the clause gadgets, connect it to the complementary variable from the variable gadget.

4.3 NAE-3-CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula φ, define G_φ as follows:

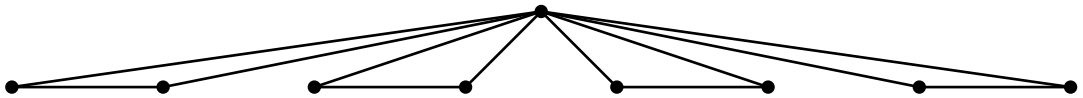
1. Start with a single vertex D. This is our *Don't care vertex*.
2. For each variable x_i of φ, add two "original" new vertices $x_i, \overline{x_i}$, add an edge between them, and connect both to D. This are our *variable gadgets*.
3. For each clause $l_1 \vee l_2 \vee l_3$ we create a triangle with 3 vertices named l_1, l_2, l_3, this is our *clause gadget*.
4. For each literal in the clause gadgets, connect it to the complementary variable from the variable gadget.

•

4.3 NAE-3-CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula φ, define G_φ as follows:

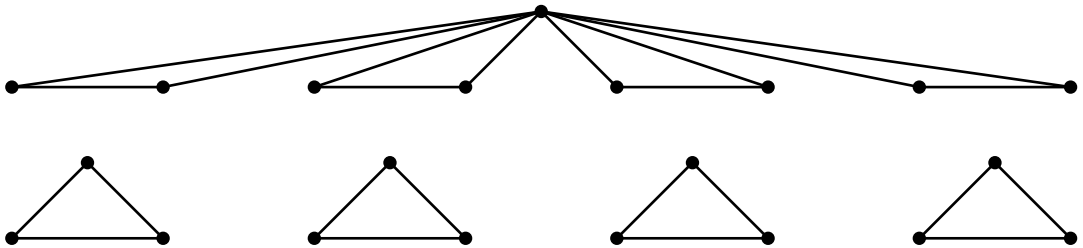
1. Start with a single vertex D. This is our *Don't care vertex*.
2. For each variable x_i of φ, add two "original" new vertices $x_i, \overline{x_i}$, add an edge between them, and connect both to D. This are our *variable gadgets*.
3. For each clause $l_1 \vee l_2 \vee l_3$ we create a triangle with 3 vertices named l_1, l_2, l_3, this is our *clause gadget*.
4. For each literal in the clause gadgets, connect it to the complementary variable from the variable gadget.



4.3 NAE-3-CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula φ, define G_φ as follows:

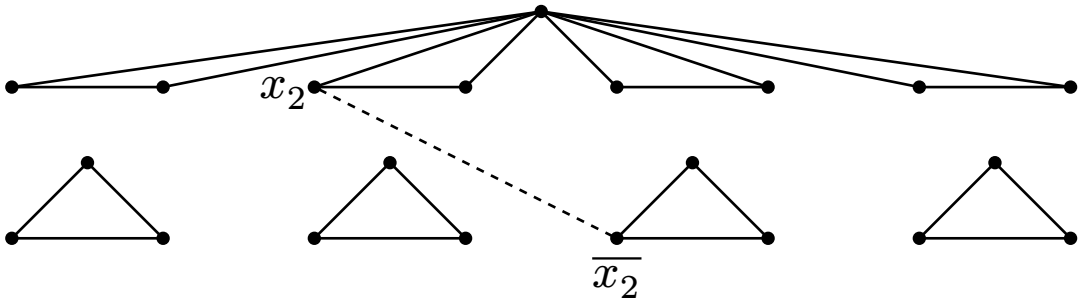
1. Start with a single vertex D. This is our *Don't care vertex*.
2. For each variable x_i of φ, add two "original" new vertices $x_i, \overline{x_i}$, add an edge between them, and connect both to D. This are our *variable gadgets*.
3. For each clause $l_1 \vee l_2 \vee l_3$ we create a triangle with 3 vertices named l_1, l_2, l_3, this is our *clause gadget*.
4. For each literal in the clause gadgets, connect it to the complementary variable from the variable gadget.



4.3 NAE-3-CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula φ, define G_φ as follows:

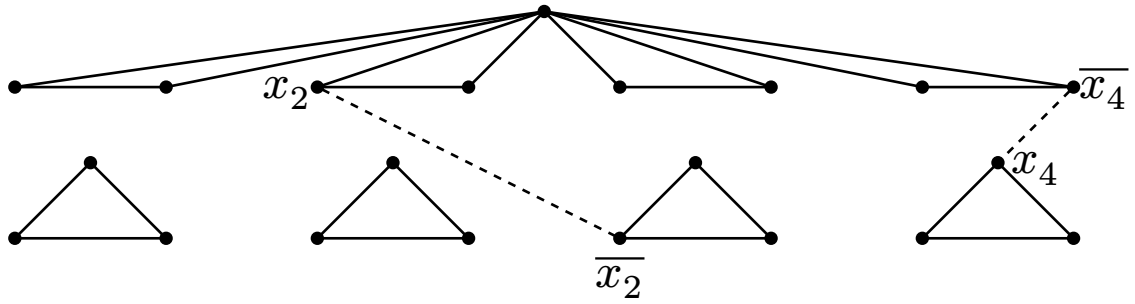
1. Start with a single vertex D. This is our *Don't care vertex*.
2. For each variable x_i of φ, add two "original" new vertices $x_i, \overline{x_i}$, add an edge between them, and connect both to D. This are our *variable gadgets*.
3. For each clause $l_1 \vee l_2 \vee l_3$ we create a triangle with 3 vertices named l_1, l_2, l_3, this is our *clause gadget*.
4. For each literal in the clause gadgets, connect it to the complementary variable from the variable gadget.



4.3 NAE-3-CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula φ, define G_φ as follows:

1. Start with a single vertex D. This is our *Don't care vertex.*
2. For each variable x_i of φ, add two "original" new vertices $x_i, \overline{x_i}$, add an edge between them, and connect both to D. This are our *variable gadgets.*
3. For each clause $l_1 \vee l_2 \vee l_3$ we create a triangle with 3 vertices named l_1, l_2, l_3, this is our *clause gadget.*
4. For each literal in the clause gadgets, connect it to the complementary variable from the variable gadget.



- The algorithm runs in poly time(why?)

4.4 NAE-3-CNF-SAT $\leq_p$ 3-COL

- We need to show that

$$\varphi \in \text{NAE-3-CNF-SAT} \Leftrightarrow G_\varphi \in \text{3-COL}$$

$\Rightarrow$:

- Given a satisfying NAE assignment α_φ for φ, define the follwing 3-coloting of G_φ:
  - D will be colored as D
  - For each "original" variable x_i, if x_i is assigned `true` under α_φ color x_i as T and $\overline{x_i}$ in F, otherwise color x_i as F and $\overline{x_i}$ in T
  - For each clause gadget, scan the corresponding clause c, color first literal that is assigned `true` with T, the first that assigned `false` with F, and color the vertex that was left with D.
- Each edge inside vertex/clause gadgets have both ends in different colors.
- W.L.O.G, let x be a variable assigned `true` by a_φ, as a_φ is proper, all vertcies $\overline{x}$ are F or D. So all edges between clause/vertex gadgets have both ends in different colors.

4.5 NAE-3-CNF-SAT $\leq_p$ 3-COL

- We need to show that

$$\varphi \in \text{NAE-3-CNF-SAT} \Leftrightarrow G_\varphi \in \text{3-COL}$$

$\Leftarrow$:

- Given a 3-coloring ψ of G_φ, we define a NAE-satisfying assignment for φ.
- As all of the variable gadgets form a triangles with a common vertex D, it leaves them with two colors to be chosen W.L.O.G, those colors are T and F.
- Assigned x_i as `true` if x_i is colored as T in its *variable gadget* otherwise assign x_i as `false`. This defines a valid assignmet to the variables of φ.
- The assignment is NAE as each clause gadget has one variable colored `true` and one `false`.

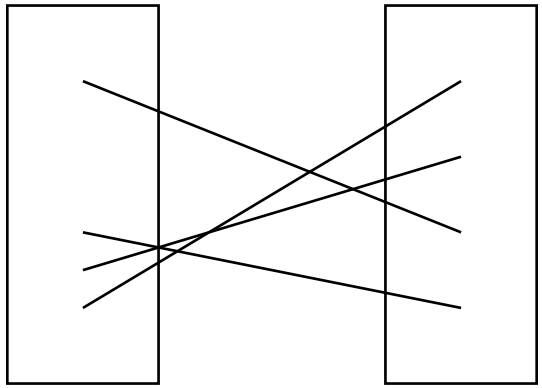
4.6 Max-cut

- Given a graph G, a *cut* is defined as the set of edges between $S \subseteq V(G)$ and $\overline{S} = V(G) \setminus S$.
- We denote the set of edges by

$$E_G\left(S, \overline{S}\right) := \left\{(u, v) : (u, v) \in E(G), u \in S, v \in \overline{S}\right\}$$

and the number of edges by

$$e_G\left(S, \overline{S}\right) := \left|E_G\left(S, \overline{S}\right)\right|$$



4.6 Max-cut

Denote by $\sigma(G) := \max_{S \subseteq V(G)} e_G\left(S, \overline{S}\right)$.

Definition 4.6.1 (MAX-CUT) MAX-CUT $:= \{< G, k >: \sigma(G) \geq k\}$

Theorem 4.6.2 MAX-CUT$\in \mathbf{NPC}$

- The proof that MAX-CUT is in **NP** is left as homework.

4.7 Max-cut

- We are going to show

$$\text{NAE-3-CNF-SAT} \leq_p \text{MAX-CUT}.$$

- Given a 3-CNF formula φ, define G_φ as follows:

4.7 Max-cut

- We are going to show

$$\text{NAE-3-CNF-SAT} \leq_p \text{MAX-CUT}.$$

- Given a 3-CNF formula φ, define G_φ as follows:

1. For each variable x_i of φ, add two new vertices $x_i, \overline{x_i}$ and an edge between them. These are our *variable gadgets.*

4.7 Max-cut

- We are going to show

$$\text{NAE-3-CNF-SAT} \leq_p \text{MAX-CUT}.$$

- Given a 3-CNF formula φ, define G_φ as follows:

1. For each variable x_i of φ, add two new vertices $x_i, \overline{x_i}$ and an edge between them. These are our *variable gadgets.*
2. For each clause $l_1 \vee l_2 \vee l_3$, we create a triangle with vertices l_1, l_2, l_3; this is our *clause gadget.*

4.7 Max-cut

- We are going to show

$$\text{NAE-3-CNF-SAT} \leq_p \text{MAX-CUT}.$$

- Given a 3-CNF formula φ, define G_φ as follows:

1. For each variable x_i of φ, add two new vertices $x_i, \overline{x_i}$ and an edge between them. These are our *variable gadgets.*
2. For each clause $l_1 \vee l_2 \vee l_3$, we create a triangle with vertices l_1, l_2, l_3; this is our *clause gadget.*
3. For each literal vertex in a clause gadget, connect it to the complementary literal vertex in the variable gadget.

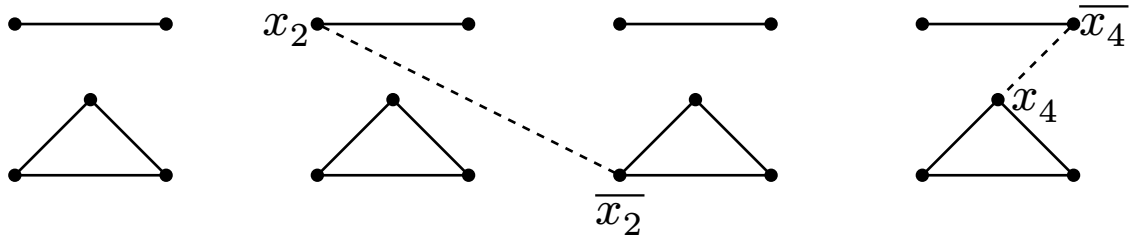
4.7 Max-cut

- We are going to show

$$\text{NAE-3-CNF-SAT} \leq_p \text{MAX-CUT}.$$

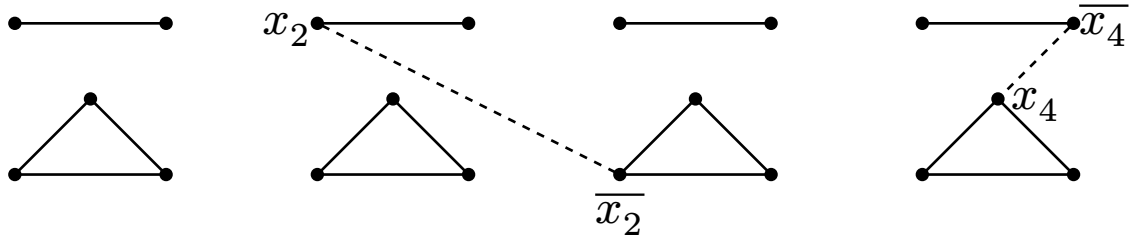
- Given a 3-CNF formula φ, define G_φ as follows:

1. For each variable x_i of φ, add two new vertices $x_i, \overline{x_i}$ and an edge between them. These are our *variable gadgets.*
2. For each clause $l_1 \vee l_2 \vee l_3$, we create a triangle with vertices l_1, l_2, l_3; this is our *clause gadget.*
3. For each literal vertex in a clause gadget, connect it to the complementary literal vertex in the variable gadget.



- The algorithm runs in poly time(why?)
- $f(\varphi)$ needs to return both the graph G_φ and a number k what should k be?

4.8 Max-cut



- What is $\sigma(\varphi)$ in the graph above?
- Intuition:
  - For each variable clause we might take 1 vertex, thus covering edges of the time $(x_i, \overline{x_i})$ thus covering exactly n edges.
  - For each triangle clause we might also take 1 or 2 vertices covering exactly 2 of the edges of each triangle thus covering $2m$ edges.
  - We are left with edges in between the *gadget variables* and *gadget clauses*, how many edges of this type we have?
    - Each literal l connect exactly to one variable $v\left(\bar{l}\right)$, so that there are $3m$ edges in between.

  So that it follows that if $\sigma\left(G_\varphi\right) \leq n + 5m$.
  - We return $< G_\varphi, n + 5m >$ and hope for the best.

4.9 Max-cut

- We need to show that

$$\varphi \in \text{NAE-3-CNF-SAT} \Leftrightarrow G_\varphi \in \text{MAX-CUT}$$

$\Rightarrow$:

- Given a satisfying NAE assignment α_φ for φ, define $S \subseteq V\left(G_\varphi\right)$ to consist of all vertices whose label is a literal assigned `true` under α_φ.
- As α_φ is consistent, all variable gadgets must cross $\left(S, \overline{S}\right)$ adding n edges to the cut.
- As α_φ is a valid NAE assignment, at least two edges cross $\left(S, \overline{S}\right)$ in every clause gadget, adding $2m$ edges to the cut.
- Each edge between a variable and clause gadget has the form $\left(l, \bar{l}\right)$, which means it is also in the cut, adding $3m$ edges to the cut.
- Overall, we count at least $n + 5m$ edges.

4.10 Max-cut

- We need to show that

$$\varphi \in \text{NAE-3-CNF-SAT} \Leftrightarrow G_\varphi \in \text{MAX-CUT}$$

$\Leftarrow$:

- Suppose that $< G_\varphi, n + 5m > \in$ MAX-CUT. Let $\left(S, \overline{S}\right)$ be a cut of G_φ such that $e_{G_\varphi}\left(S, \overline{S}\right) = n + 5m$.
- Define the assignment α_φ for φ in which all variable gadget literals found in S are assigned `true` and all remaining are assigned `false`. This defines a consistent assignment.
- It remains to prove that α_φ is NAE-satisfying.
  - Fix a clause of φ, and look at the corresponding gadget clause.
    - Since $e_{G_\varphi}\left(S, \overline{S}\right) = n + 5m$, every edge between this clause gadget and the variable gadgets is also in the cut and of the form $\left(l, \bar{l}\right)$.
    - Take a vertex l in that clause gadget that is also in S. Then the edge $\left(l, \bar{l}\right)$ implies that $\bar{l} \in \overline{S}$, meaning the literal corresponding to l is assigned `true`. In a similar manner, if l is in $\overline{S}$ the then the edge $\left(l, \bar{l}\right)$ means the $\bar{l} \in S$ implying that l is assigned `false`.
    - On the otherhand since $e_{G_\varphi}\left(S, \overline{S}\right) = n + 5m$, each clause gadget has 2 edges crossing $\left(S, \overline{S}\right)$, meaning that this clause gadget has at least one vertex in S and one in $\overline{S}$.